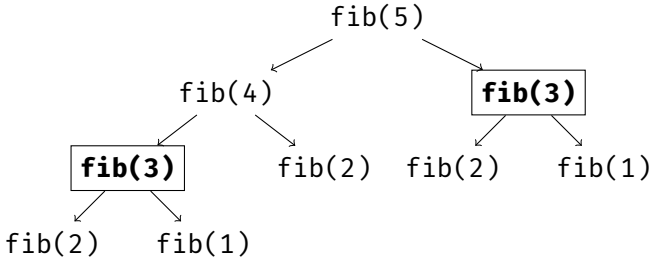
6 Remember Me! (15pts)

6.1 Motivation (3pts)

We, programmers, write recursive functions to solve recursive problems. As an example, to compute the nth number of the classic Fibonacci sequence, we can directly translate the recursive definition of the Fibonacci sequence into a naive recursive function named `fib(n)`.

$$F_n = \begin{cases} 1, & n = 1, 2 \\ F_{n-1} + F_{n-2}, & n \geq 3 \end{cases} \tag{1}$$

```
1 def fib(n):
2     if n == 1 or n == 2:
3         return 1
4     else:
5         return fib(n - 1) + fib(n - 2)
```



However, the function `fib` is not efficient. For example, to evaluate `fib(5)`, python will have to evaluate `fib(3)` twice, one for `fib(4)` and one for `fib(5)`. Suppose `fib(1)` and `fib(2)` return immediately and do not make subsequent calls to `fib` as they are the base cases for computing the Fibonacci sequence, write a function `count_calls(n)` to count the number of function calls to `fib` that is needed to evaluate `fib(n)`. To give an example, `count_calls(5)` returns 9 since python needs to evaluate the function `fib` 9 times to compute `fib(5)`.

```
1 def count_calls(n):
2     if ____________________:
3         return 1
4     return ________________________________________________
```

According to `count_calls`, the running time of the function `fib` grows _____ as the input argument `n` increases.

(A) Constantly
(B) Linearly
(C) Logarithmically
(D) Exponentially

To make our lives easier, we definitely want to speed up the process of computing `fib(n)`. But before digging into details, we will have a closer look at the function `fib` we are going to optimize to identify some useful properties of it. `fib` is a pure function, it is recursive, it has no side effects, it takes exactly one argument, etc. But among these properties, the most important is that it is a so-called *pure* function.

6.2 Pure (4pts)

Simply put, a function is **pure** if it always returns the same value when given the same arguments and has no side effects (i.e., changing some global states). With pure functions, we can remember the function's results for some given inputs. When the function is called next time with the same inputs, we can immediately return what we remembered since the results remain the same, and we need not worry about missing side effects even though we skipped the function's body.

For each of the functions in the table, write **T** if it is a pure function, write **F** otherwise. If the function has an input argument, you can assume that the argument will always be **positive integers**.

Function	Your Answer
`song = lambda : 'APT'`	**T**

`import random`
`def lottery():`
`  return random.randint(6, 71)`	

`def fact(n):` `  return 1 if n == 0 else n * fact(n - 1)`	

`t = []`
`def append_len(x):`
`  t.append(x)`
`  return len(t)`	

`def fun(x):`
`  def safe(y, total):`
`  nonlocal x`
`  if y <= 0:`
`  return total`
`  x = (y - 1) * 2`
`  return safe(y - 1, x * (total + 1))`
`  return safe(x, 0)`	

6.3 Rewrite Me? (2pts)

For a pure function with only one argument `n`, we can always write another function to do the same thing as the original function does, except that the new function will *remember* the result `Fn` for a given `n` after computation, and *recall*s the *remember*ed result next time immediately if it has *remember*ed the result for that same argument `n`. Figuratively speaking, we can write a function that *learn*s from its past!

Now complete function `fib_r`, the improved version of `fib`.

```
1 mem = memory()
2 def fib_r(n):
3     if ____________________:
4         return recall(mem, n)
5     Fn = 1
6     if ___________________:
7         Fn = fib_r(n - 1) + fib_r(n - 2)
8     remember(mem, n, Fn)
9     return Fn
```

Here, we use an ADT called `memory` to

- `memory()`
  create a new memory that remembers nothing
- `remember(mem, n, Fn)`
  remember that the `n`th element of the sequence is `Fn`, and store this piece of information in memory `mem`.
- `recall(mem, n)`
  return the `n`th element of the sequence if it is remembered, returns **None** if not.

`fib_r` seems nice, and surely does a great job in improving the efficiency of `fib`. However, having to write a completely *new* function to solve an already solved task is laborious, as we are lazy programmers and are unwilling to rewrite existing code (aren't we all?).

6.4 Remember Me! (6pts)

To go one step further, let's write a function `remember_me(f)` to produce an improved version of *any* given function `f` so that we do not have to write functions over and over again to speed up existing functions.

```
def remember_me(f):
    _________________
    def inner(n):
        if _________________:
            return recall(mem, n)
        ________________________
        ________________________
        return ___________________
    return inner
```

After finishing `remember_me`, we can simply use one assignment statement to bind the variable `fib` to the improved version created by `remember_me`. Later call expressions whose operator is `fib` will then call the efficient remembered version of the original `fib` function. We give an example here to demonstrate the power of `remember_me` by computing the 100-th number of the Fibonacci sequence.

```
>>> fib = remember_me(fib)
>>> fib(100) # This is huge
354224848179261915075
```

Moreover, `remember_me` enables us to transform *any* pure function with one argument into an efficient remembered version, using only one assignment statement (the power of abstraction, again!). And as we will soon see, the restriction on the number of arguments is not really that big a problem since we can do some clever encoding and decoding to trick the Python interpreter.

For pure functions with more than one argument, we can always pack the arguments into a single tuple to mimic a single-argument function. We use the function `count_k` from lab03 as a demonstration.

```
def count_k(n, k):
    """Counts the number of paths to climb up a flight of n stairs,
    taking up to and including k steps at a time."""
    if n == 0:
        return 1
    total, i = 0, 1
    while i <= min(n, k):
        total += count_k(n - i, k)
        i += 1
    return total
```

For `count_k`, we can rewrite it to accept a single tuple as the input argument.

```
def count_k_1(nk):
    """Counts the number of paths to climb up a flight of n stairs,
    taking up to and including k steps at a time."""
    n, k = nk
    if n == 0:
        return 1
    total, i = 0, 1
    while i <= min(n, k):
        total += count_k_1((n - i, k))
        i += 1
    return total
```

```
>>> count_k_1 = remember_me(count_k_1)
>>> count_k_1((100, 2)) # Also huge
573147844013817084101
```

Again, since we are all lazy programmers, can we write another function, say, `two_to_one(f)`, to transform a pure function `f` with two arguments `n` and `k` into another function that behaves exactly the same, except that the transformed function uses a tuple `(n, k)` to encode the two arguments of the function `f`?

Here is an attempt to do so. Figure out what would interactive Python interpreter display after evaluating the given expression.

```
def two_to_one(f):
    return lambda nk: f(nk[0], nk[1])
```

```
>>> count_k = two_to_one(count_k)
>>> count_k((5, 3))
_________________
```

7. (21 points) Automatic Function Composition

In our half semester of programming learning, we often use given functions and variables to write a program, in order to complete some task. Well, have you ever thought of throwing these functions and variables to your computer and urging it to automatically write the program itself? Although it sounds crazy, we are now going to implement a toy version!

Automatic Function Composition Problem

Given: We are given an integer pair (x, y) (also called input-output pair), a list of defined functions $[f_1, f_2, \dots, f_n]$ (also called function list), and a list of integers $[t_1, t_2, \dots, t_n]$ (also called times list), where each function f_i $(i = 1 \dots n)$ takes exactly one integer argument and returns exactly one integer value.

Goal: We want to automatically find a non-empty list of m functions $[f_{i_1}, f_{i_2}, \dots, f_{i_m}]$ (where $m > 0$, $i_j \in \{1, \dots, n\}$, for each $1 \le j \le m$) where each function is chosen from the original n functions, and compose these chosen ones as $g = \text{lambda } x\text{: } f_{i_m}(f_{i_{m-1}}(\dots(f_{i_1}(x))))$, which satisfies that $g(x)$ equals y. Note that each function $\mathrm{f_i}$ in the original function list can only be chosen at most $\mathrm{t_i}$ ($\mathrm{t_i}$ is an integer greater than 0) times and at least 0 times (0-times means not chosen). For example, $[f_1, f_1]$ would be a solution to the problem with input-output pair (x_1, y_1), function list $[f_1, f_2, f_3]$, times list $[3,3,3]$, if $f_1(f_1(x_1))$ equals y_1. In this example, f_1 is chosen twice (2 times), while both f_2 and f_3 are not chosen (0 times). These chosen times ($2, 0, 0$ times) are all in the range from 0 to 3.

The following two examples help you understand this problem.

Example 1: We are given an input-output pair (i.e., 2-tuple) `(4, 6)`, a function list `[dec, mul2]`, and a times list `[2, 2]`, where `dec = lambda x: x-1` and `mul2 = lambda x: 2*x`. A solution is `[mul2, dec, dec]`, which means the composed function `g = lambda x: dec(dec(mul2(x)))` makes `g(4) == 6` true. However, `[dec, mul2]` is also a solution, since `mul2(dec(4)) == 6` is true. The example suggests that we may have one or more solutions to the automatic function composition problem. In fact, the solution list to this problem is `[[dec, mul2], [mul2, dec, dec]]`.

Example 2: We are given an input-output pair `(4, 6)`, a function list `[mul2, mul3]`, and a times list `[1000, 1000]`, where `mul2 = lambda x: 2*x` and `mul3 = lambda x: 3*x`. We cannot find any solution to this concrete problem (the solution list is `[]`), which suggests that we may have no solutions to the automatic function composition problem. No need to be surprised because the given function list and times list restrict the power of composing the program you want.

(a, 4 points) Do the following problems have solutions? If so, write ALL the solutions in a list (each solution is a function list looking just like `[mul2,dec,dec]` in Example 1, and the order between solutions does not matter). If not, just write "No solution".

(1) Input-output pair: `(11, 3)`
Function list: `[div2, inc]`, where `div2 = lambda x: x // 2` and `inc = lambda x: x + 1`
Times list: `[2,1]`

(2) Input-output pair: `(5, 1)`
Function list: `[div3, add3]`, where `div3 = lambda x: x // 3` and `add3 = lambda x: x + 3`
Times list: `[1,1]`

(b, 12 points) Implement the functions `satisfy` and `afc` whose frameworks are shown below. `afc` takes in an input-output pair `io_pair`, a function list `func_list`, and a times list `times_list` with integer elements. `afc` returns a list storing ALL the solutions (the order between solutions does not matter) to the corresponding automatic function composition problem with these given arguments. If it had no solution, the returned solution list would be an empty list. You may use the function `satisfy` in your implementation of `afc`.

```
def satisfy(io_pair, sol):
    """Returns True if sol satisfies io_pair, otherwise returns False.
    """
    input_x = io_pair[0]
    output_y = io_pair[1]
    eval_result = _____________________________
    for _____________________________:
        eval_result = _____________________________
    return _____________________________________

def afc(io_pair, func_list, times_list):
    """Returns the solution list of the automatic function composition problem
    given io_pair, func_list, and times_list.
    """
    assert len(func_list) == len(times_list)
    solutions = []

    def sol_search(remain_times_list, cur_sol):
        # cur_sol is a list of currently chosen functions
        # Every solution should NOT be empty
        if _____________________________:
            solutions.append(cur_sol)
        for _____________________________:
            if _____________________________________________:
                new_times_list = ____________________________________________________________
                sol_search(new_times_list, _____________________________________________)

    sol_search(times_list, [])
    return solutions
```

(c, 5 points) From now on, this problem will be extended a bit. Input-output pair is now extended to input-output pair list $[(x_1, y_1), (x_2, y_2), \ldots, (x_p, y_p)]$, which is a list containing p input-output pairs. Now a solution g to this extended problem should satisfy that FOR EACH input-output pair (x_i, y_i) (where $i \in \{1, \ldots, p\}$), we have $g(x_i)$ equals y_i.

Example 3: We are given an input-output pair list `[(2,0), (3,3)]`, a function list `[dec, mul3]`, and a times list `[2, 2]`, where `dec = lambda x: x-1` and `mul3 = lambda x: 3*x`. The solution list to this problem is `[[dec, dec, mul3]]`, since `mul3(dec(dec(2))) == 0` and `mul3(dec(dec(3))) == 3`. Although `[dec, dec]` satisfies the input-output pair `(2, 0)`, it fails to satisfy `(3, 3)`, leading to its absence from the final solution list.

Please implement the function `extended_afc`, where you should use the functions `afc` and `satisfy` defined in question (b).

```
def extended_afc(io_pair_list, func_list, times_list):
    """Returns the solution list of the automatic function composition problem
    given io_pair, func_list, and times_list.
    """
    assert len(io_pair_list) > 0
    solutions = ____________________________________________________________________
    return my_filter(_____________________________________________________, solutions)
```

Hint:

(1) You only need to use `afc` once.

(2) You may need to use `my_filter` function and `my_all` function. Their implementations are shown below.

```
def my_filter(f, lst):
    return [element for element in lst if f(element)]

def my_all(lst):
    for bool_elem in lst:
        if not bool_elem:
            return False
    return True
```